University of Cape Town



Machine Learning Assignment 1

Pamela Afful

Student Number: AFFPAM001

October 18, 2026

Department of Statistical Sciences Plagiarism Declaration form

A copy of this form, completed and signed, to be attached to all coursework submissions to the Statistical Sciences Department. Submissions without this form will not be marked.

COURSE CODE: **STA** 5068Z

COURSE NAME: MACHINE LEARNING

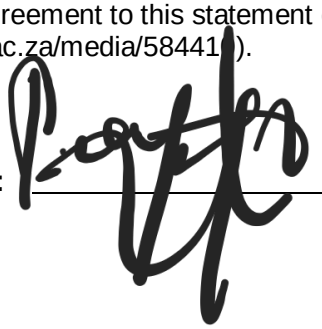
STUDENT NAME: PAMELA AFFUL

STUDENT NUMBER: AFFPAM001

PLAGIARISM DECLARATION

1. I know that plagiarism is wrong. Plagiarism is to use another's work and pretend that it is one's own.
2. I have used a generally accepted citation and referencing style. Each contribution to, and quotation in, this tutorial/report/project from the work(s) of other people has been attributed, and has been cited and referenced.
3. This tutorial/report/project is my own work.
4. I have not allowed, and will not allow, anyone to copy my work with the intention of passing it off as his or her own work.
5. I acknowledge that copying someone else's assignment or essay, or part of it, is wrong, and declare that this is my own work.

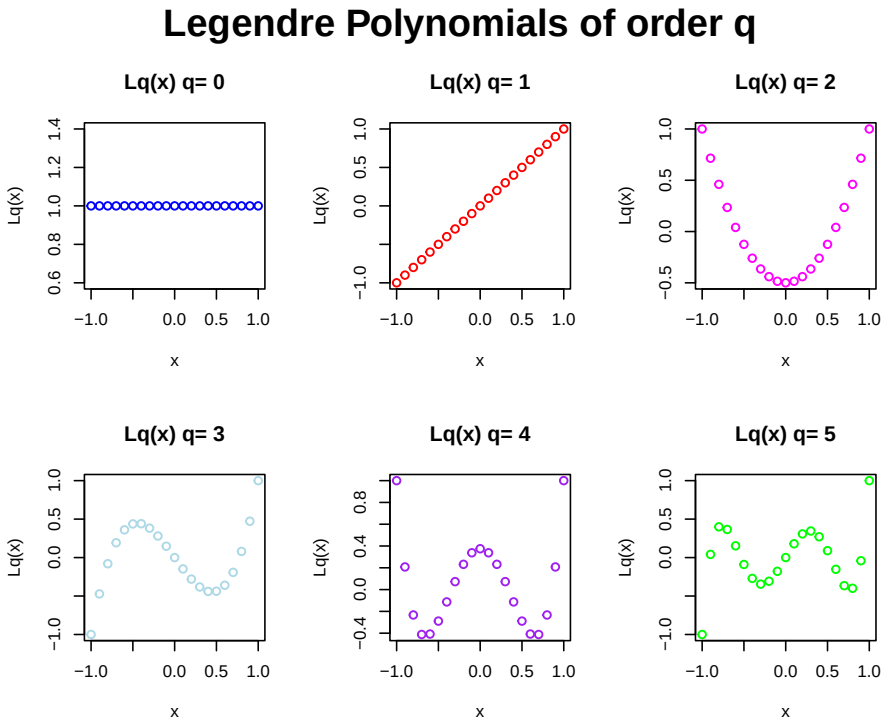
Note that agreement to this statement does not exonerate you from UCT's Academic Misconduct policy (https://uct.ac.za/media/58441).

Signature: 

Date: 18/10/2026

Question 1

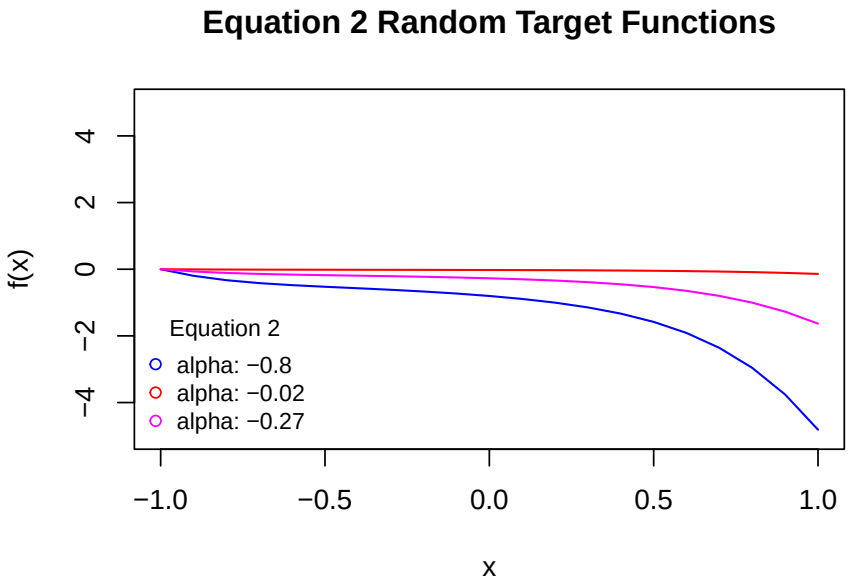
a) Legendre Polynomial Plots

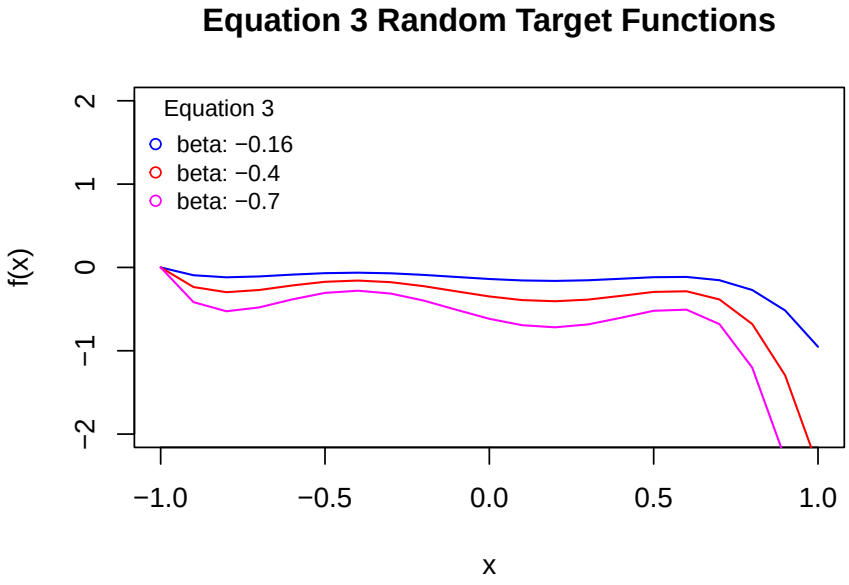


As the order of the polynomial, q, increases, the more nonlinear/complex the $L_q(x)$ becomes:

1. At q=0, $L_q(x)$ devolves to a straight horizontal line $L_q(x)$ =1 over the domain [-1,1].
2. At q=1, $L_q(x)$ starts to have some slope attached, a straight line of the form $y = 2x$.
3. At q=2 the non-linear curvature/complexity of $L_q(x)$ starts to become more visible, particularly towards the edges of the domain [-1,1]; the function approximates a parabola/quadratic function.
4. At q=3,4,5, the non-linear, polynomial curvature/complexity of $L_q(x)$ becomes more prominant with the function taking more sinusoidal shape.

b) Random Target Functions



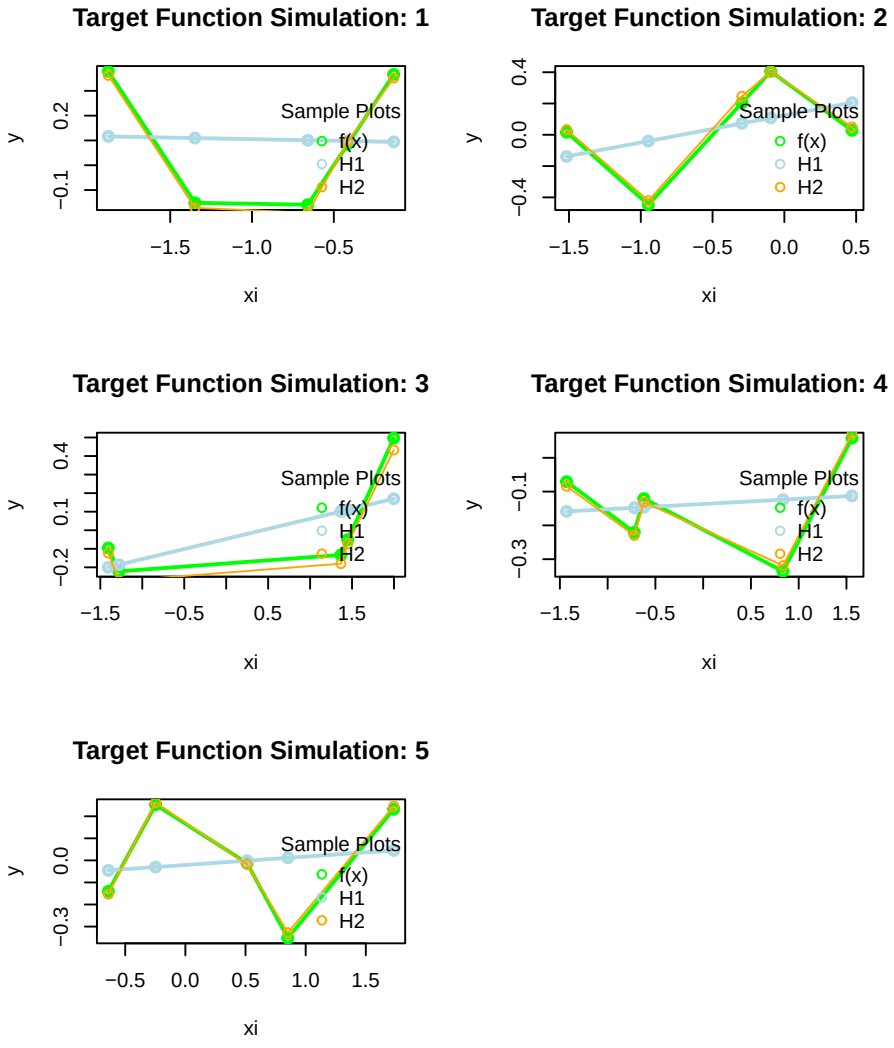


Observations:

1. $f(x) = \sum_{q=0}^{Q_f} \alpha_q x^q$: the functions only output values from 0 to negative values over the domain [-1,1], although the lines are smoother with very little curvature in between.

2. $f(x) = \sum_{q=0}^{Q_f} \beta_q L_q(x)$: the functions also output 0 to negative values over the domain; however, over the domain [-1,1], however, for certain values of β there are curvatures, with the function taking a wave-like shape.

Question 2



b) A Priori expectation

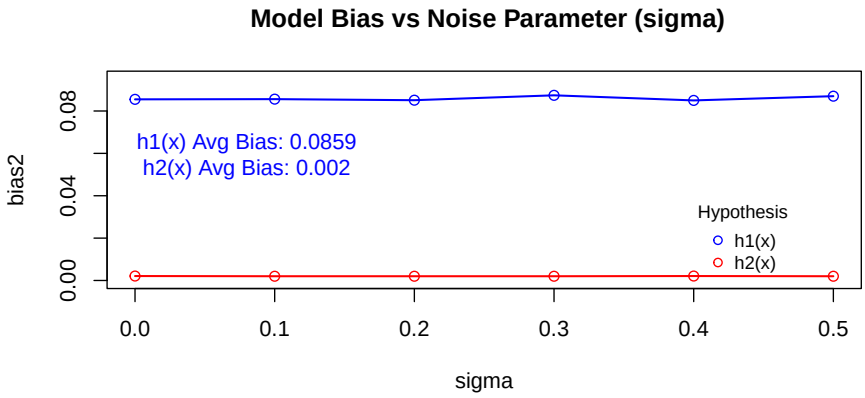
Given that the underlying data generating process is non-linear, the non-linear hypothesis would be more suited to fit the data's non-linear behavior and overall shape. This visually seems to be the case as well: the H2(x) hypothesis functions follows the shape of the data quite closely. The linear function only fits a straight line through the data with only a few values intersecting with the underlying

function output values.

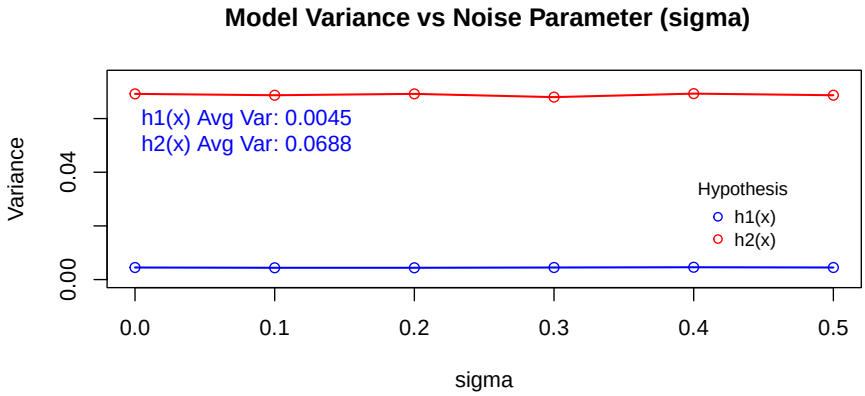
c) Bias Variance for H1 and H2

	Bias	Variance	Both:Bias+Var	OOS Error	Estimate Differance
h1(x)	0.0854	0.0044	0.0898	0.0841	0.0057
h2(x)	0.0021	0.0709	0.0730	0.1295	0.0565

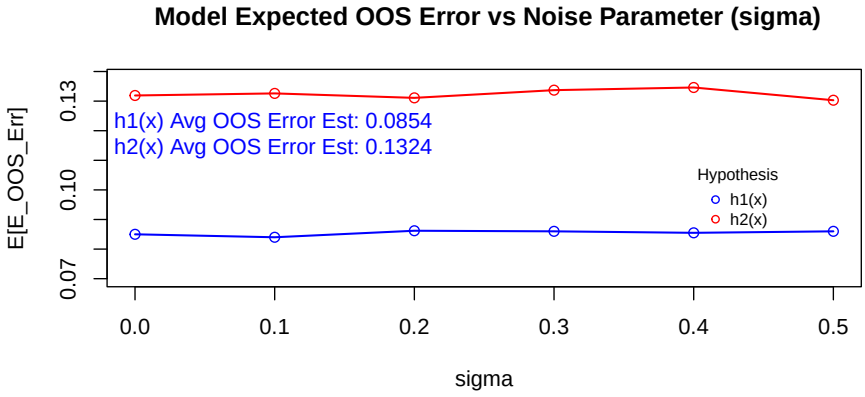
d) Bias Variance Dynamics



(a) Bias vs Noise



(b) Variance vs Noise



(c) E[OOS Error] vs Noise

Figure 1: OOS Error Estimates

e) Interpretation

Bias-Variance Dynamics

The expected out-of-sample error as it relates to the bias and variance of a model is given by

$$E[Err_{oos}] = Bias^2 + Variance + IrreducibleNoise(\varepsilon)$$

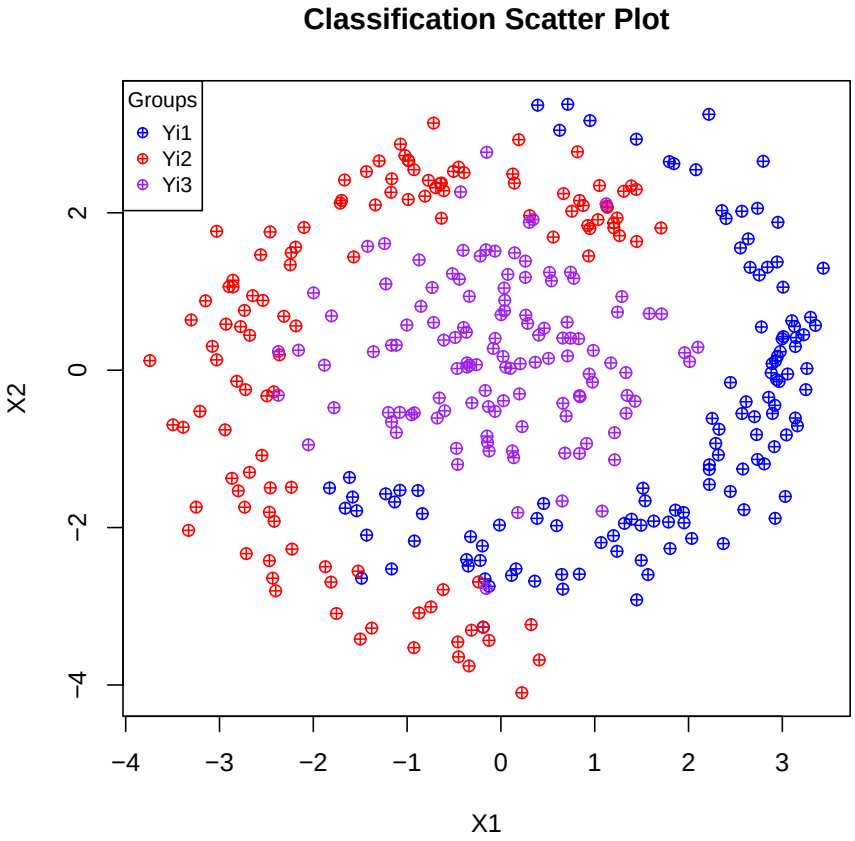
- RHS:
  - The linear model, $h1(x)$, has better out-of-sample performance overall, averaged across 1000 simulations over varying values of the noise parameter, compared to the more complex model, $h2(x)$.
  - There are relatively small changes in the average OOS error for both hypothesis functions. The steepest increase in the average OOS error is observed in the change of the noise parameter from 0.1 to 0.2 and levels thereafter for $h1(x)$ and mostly increases for $h2(x)$ as the noise parameter increases from 0.2.
- LHS:
  - There is very little variability/ change observed in the variance as the noise parameter increases for both functions, whilst the bias for $h1(x)$, sees a small decrease when the noise level increases from 0.3 to 0.4.
  - Overall, the linear model, $h1(x)$, has lower average variance but higher bias across the various noise parameters; this is expected given the that the model is less complex relative to $h2(x)$.

Model Choice

- Given the estimated OOS Error, it's better to go with the linear model as it is more parsimonious and more likely to be able to generalize to new data as observed across the above simulations.
- Rational: the combined estimate comparison $bias^2 + Var$ to $E[Err_{oos}]$ across sigmas :
  - $h1(x)$: its average RHS, 0.0904, is closer to its LHS OOS estimate, 0.0854, providing a better estimate of the model's general OOS behavior compared to $h2 = (x)$; its average RHS, 0.0708 has a much larger deviation from its average LHS estimate, 0.1323.

Question 4

a) Data Set Plot



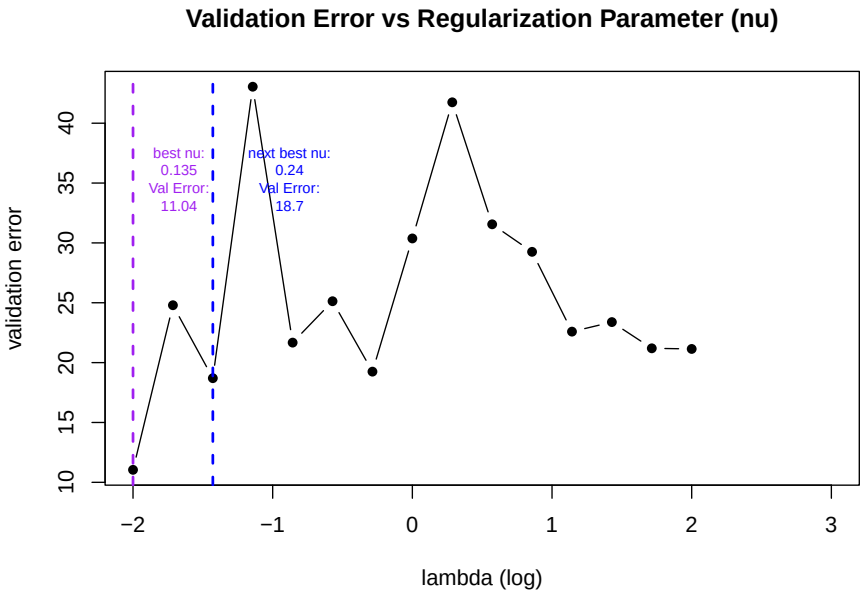
Observation: The data does not appear to be linearly separable (in the given 2D feature space) as the data does not seem to have linear boundaries between categories/classes. Instead, the different classes of responses look to be spiraled around each other. A non-linear neural network seems well suited to capture the non-linear pattern/structure embedded in the data for the given feature space.

c) Feed Forward Neural Network

Numerical Pitfalls: The computation of $log(\hat{y})$ in the objective function may cause the network to explode/ diverge given the undefined behavior of the log function at 0 (or values very close to 0), which can hamper the numerical stability of the network's computations.

d) Validation Analysis

Plot the validation error vs log (λ)

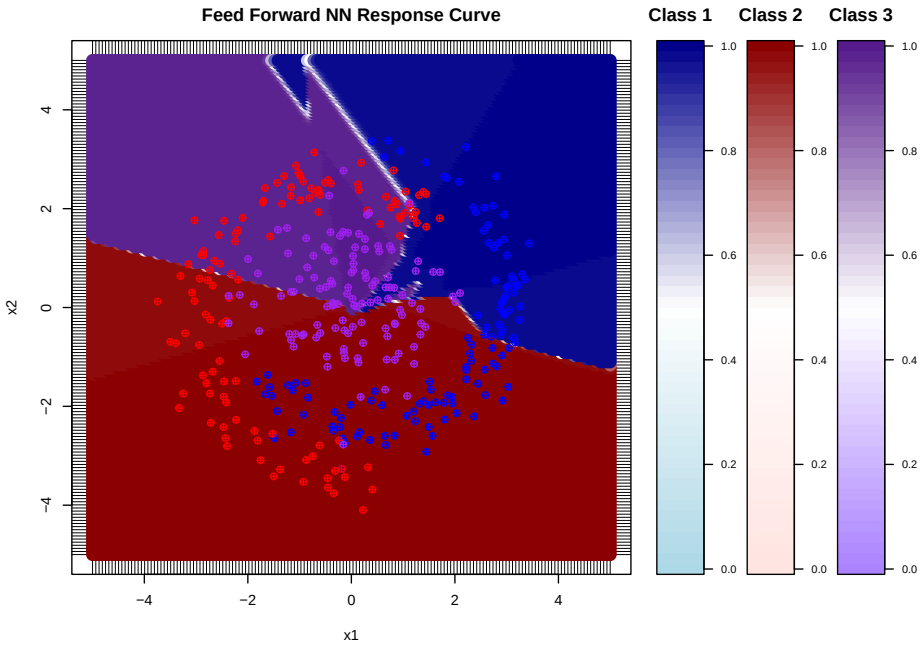


Model Parameters: number of hidden layer nodes, **m =10**

The lowest validation error is generated at $\lambda_{min} = 0.135$ with an associated validation error of 11.04, making for a less strict and more complex model (given the big budget that the sum of the model's parameters cannot exceed). The next best error to the right of this minimum is obtained at $\lambda_{1SE} = 0.24 (Val_{err} = 18.7)$, offering stricter control over the complexity of the network and thus more parsimonious model, decreasing the likelihood of the model over fitting to just the training data. **This is the better choice of regularization parameter.**

d) Response Curve

Model Parameters: **m=10, nu=0.24**



The model produces three distinct, somewhat jagged regions over the feature space, each corresponding to the probability assignments for one class:

- Class 1: Roughly over half of the Class 1 observations are misclassified. The upper-right region captures only some of them, while a cluster near the bottom (X1 between about -2 and 3, X2 between about -3 and -1.5) falls within the Class 2 region. The model is very confident within its Class 1 region, assigning probabilities close to 1, so new observations resembling that bottom cluster would likely be misclassified as Class 2.

- Class 2: Class 2 predictions dominate the feature space. The region lies below a boundary that slopes from X2 $\sim$ 1.4 at (X1 = -5) down to $\sim$ -1.2 (at X1 = 5), and the model predicts Class 2 there with probabilities close to 1. However, a significant portion of Class 2 observations (X1 between about -3 and 1, X2 between about 1.5 and 3) falls within the Class 3 region.

- Class 3: The Class 3 region extends from the left edge (X1 = -5) to a diagonal boundary running from about X1 = -1 at the top to about X1 = 1 near X2 = 0.5, and it narrows toward the top. Roughly half of the Class 3 observations lie outside this region, mostly in the Class 2 region, and new observations would likely behave similarly. Probabilities within the region are again high.

Appendix

Question 1 code

```
lq=function(x,q){
  coeff=2**q
  summands_store=c()
  for(k in 0:q) { # These are the ks we are summing over

   summands_k=(x**k)*(choose(q,k)*choose(((q+k-1)/2),q)
 ↪  )
   summands_store=c(summands_store,summands_k)
  }

  f_x=coeff*sum(summands_store)

  return(f_x)

}




# plot the above
# USE sapply to vectorize the calculation across a
 ↪  supplied vector
x_list=list(x=seq(-1,1,0.1))

x=seq(-1,1,0.1)
Qs=seq(0,5,1)

q_cols=c('blue','red','magenta','lightblue','purple','green')



par(mfrow = c(2, 3),oma = c(0, 0, 3, 0))

for(i in 1:length(Qs)){
 ↪  #c(min(sapply(x,lq,q=5)),max(sapply(x,lq,q=5)))
  q_i=Qs[i]

   ↪  plot(x,sapply(x,lq,q=q_i),col=q_cols[i],lwd=1.2,xlab='x',ylab='Lq(x)',mai
   ↪  paste('Lq(x) q=',q_i))
```

```
}

mtext("Legendre Polynomials of order q", side = 3, outer
 ↪  = TRUE, cex = 1.5, font = 2)

#ya
# plots


#| label: fig-subplots
#| fig-cap: "f(x) two ways"
#| fig-subcap:
#|   - "f(x): Equation 2"
#|   - "f(x): Equation 3"
#| layout-ncol: 2


set.seed(30)

alpha_qs=runif(3,-1,1)
beta_qs=runif(3,-1,1)

target_alpha=function(x,alpha_q,Qf){
  store_q=c()
  for(q_i in 0:Qf){

    store_q=c(store_q,alpha_q*(x**q_i))
  }

  ret=sum(store_q)
  return(ret)
}


target_beta=function(x,beta_q,Qf){
  store_q=c()
  for(q_i in 0:Qf){
    Lq=lq(x,q_i)
    store_q=c(store_q,beta_q*Lq)
  }

  ret=sum(store_q)
```

```
  return(ret)
}


# Plot the above target_alpha

plot(NULL,xlim=c(-1,1),ylim=c(-5,5),xlab='x',ylab='f(x)',main='Equation
 ↪  2 Random Target Functions')

q_cols=c('blue','red','magenta','lightblue','purple','green')

for(i in 1:length(alpha_qs)){
  alpha=alpha_qs[i]


   ↪  lines(x,sapply(x,target_alpha,alpha_q=alpha,Qf=5),col=q_cols[i],lwd=1.2)
}


legend("bottomleft",
       legend = c(
        ↪  paste('alpha:',round(alpha_qs[1],2)),paste('alpha:',round(alpha_qs[2
        ↪  # Text labels
       col = q_cols[1:3],
       pch = 1,
       cex=0.85,
       title = "Equation 2",
       bty='n')

# Plot the above target_beta


plot(NULL,xlim=c(-1,1),ylim=c(-2,2),xlab='x',ylab='f(x)',main='Equation
 ↪  3 Random Target Functions')

for(i in 1:length(beta_qs)){
  beta_i=beta_qs[i]

   ↪  lines(x,sapply(x,target_beta,beta_q=beta_i,Qf=5),col=q_cols[i],lwd=1.2)
}


legend("topleft",
```

```
       legend = c(
        ↪  paste('beta:',round(beta_qs[1],2)),paste('beta:',round(beta_qs[2],2)
        ↪  # Text labels
       col = q_cols[1:3],
       pch = 1,
       cex=0.85,
       title = "Equation 3",
       bty='n')
```

Question 3 code

```
set.seed(2023)
f_x=function(x,sig=0){
  epsi=rnorm(1,0,sig)

  if(x<0){
    target=abs(x+1)-0.5
    return (target+epsi)

  }
  if(x>=0){
    target=abs(x-1)-0.5

    return(target+sig)

  }
}

# Simulte 5 time
n_sim=seq(1,5,1)
N=5

# plot(NULL, axes = FALSE,
 ↪  xlim=c(-2,2),ylim=c(-0.5,0.5),xlab='xi',ylab='f(xi)',main='Target
 ↪  Function Simulation')
# axis(1, pos = 0)
# axis(2, pos = 0)

par(mfrow = c(3,2 ))
colours=c('green','blue','lightblue','orange','red')
for(i in seq_along(n_sim)){
  xi=sort(runif(N,-2,2))
  y=sapply(xi,f_x)
```

```
  plot( y~xi,lwd=2,col=colours[1],type='l',
    ↪  main=paste('Target Function Simulation:',i))
  points(y~xi,lwd=2,col=colours[1],pch=1)

  # Plot Hypotheses
  h1=lm(y~xi)
  h1_predict=unname(predict(h1,data.frame(xi)))
  lines(h1_predict~xi,lwd=2,col=colours[3])
  points(h1_predict~xi,lwd=2,col=colours[3],pch=1)
  #
  h2=lm(y~sin(pi*xi)+cos(pi*xi)-1) # no coeff regression
  h2_predict=unname(predict(h2,data.frame(xi)))
  lines(h2_predict~xi,lwd=1,col=colours[4])
  points(h2_predict~xi,lwd=1,col=colours[4],pch=1)
  #

  legend("bottomright",
       legend = c("f(x)", "H1", "H2"), # Text labels
       col = c(colours[1],colours[3],colours[4]),
        ↪
       pch = 1,
       cex=0.95,
       title = "Sample Plots",
       bty='n')

}




# Funtion Arg
# f_tru: True DGP
# hyp: Hypothesis function
# sig: nr of std deviation from mean
# N: size of data
# nsim= nr of simulations/draws

oos_estimate=function(f_true,hyp,sig,N, nsim=1000){

  #RHS BV trade-off
  bias_store=c()
  prediction_sim_store=matrix(NA,nrow=N,ncol=nsim) #
 ↪  store predictions for each simulation in this
 ↪  matrix,col= prediction at simulation i
```

```
 pred_var_score=c()
 oos_err_store=c()
 for(i in 1:nsim){
     xi=sort(runif(N,-2,2)) # sample N samples
     y=sapply(xi,f_x) # apply the f_true

     h_predict=unname(predict(hyp,data.frame(xi))) #
↪  get prediction of the hypothesis on the sample

     # Calculate the bias at draw i
     bias_i= (y-h_predict)**2
     bias_store=c(bias_store,bias_i) # store the bias
↪  at i

     # Store predictions
     prediction_sim_store[,i]=h_predict # at each
↪  column, insert the prediction

     # get running mean of the prediction
     # use sapply for the first simulation draw

↪  mean_prediction=ifelse(i>1,apply(prediction_sim_store[,1:i],1,mean),sapply
↪

     # how far each prediction is from the mean
      ↪  prediction
      pred_var_i=(h_predict-mean_prediction)**2
      pred_var_score=c(pred_var_score,pred_var_i)


   #LHS
      # create OOS datapoint
      xi_oos=sort(runif(N,-2,2)) # draw at random again
      y_oos=sapply(xi_oos,f_x)

      xi_oos_df=data.frame(xi=xi_oos)

      h_predict_oos=unname(predict(hyp,data=xi_oos_df))

      # calculate the error SE^2
      oos_err_i=(h_predict_oos-y_oos)**2
      oos_err_store=c(oos_err_store,oos_err_i)
```

```
   }

  #  get the averages
  model_bias2=round(mean(bias_store),4)
  model_var=round(mean(pred_var_score),4)
  both=round((model_bias2+model_var+sig),4)
  oos_err_est=round(mean(oos_err_store),4)
  estimate_diff=round(abs(both-oos_err_est),4)
  both_with_noise=both+sig



   ↪  return(list(model_bias2=model_bias2,model_var=model_var,both=both,oos_err
}


# Test the above
f_true=f_x
sig=0
N=5



h1_out=oos_estimate(f_true,h1,sig,N, nsim=1000)

h2_out=oos_estimate(f_true,h2,sig,N, nsim=1000)


comaprison_table=rbind(data.frame(h1_out),data.frame(h2_out))
 ↪  %>%
  mutate(model=c('h1(x)','h2(x)')) %>%
  column_to_rownames(var = "model")

kable(comaprison_table, col.names =
 ↪  c('Bias','Variance','Both:Bias+Var','OOS
 ↪  Error','Estimate Differance'), align = "cc")
```

Question 4 code

```
dat = read.table('Collider_Data_2022.txt', h =
 ↪  TRUE,stringsAsFactors =T)
```

```
plot(NULL,
 ↪  xlim=c(min(dat['X1']),max(dat['X1'])),ylim=c(min(dat['X2']),max(dat['X2'])),
 ↪  xlab='X1',ylab='X2',main='Classification Scatter
 ↪  Plot')

points(dat[dat$Yi1==1,]$X1,dat[dat$Yi1==1,]$X2,col='blue',pch=10)

points(dat[dat$Yi2==1,]$X1,dat[dat$Yi2==1,]$X2,col='red',pch=10)

points(dat[dat$Yi3==1,]$X1,dat[dat$Yi3==1,]$X2,col='purple',pch=10)

legend(x='topleft',
       legend = c("Yi1", 'Yi2','Yi3'),
       col=c('blue','red','purple'),
       pch=c(10,10,10),
       title = "Groups",
       cex=0.85,
       bty = "o")


soft_max = function(z) {  # assume z is a matrix

  z_shifted = sweep(z, 2, apply(z, 2, max), FUN = "-") #
 ↪  subtract maximum from each column to avoid default R
 ↪  behaviour when adding numbers of varying magnitudes


  num = exp(z_shifted)

  # Sum exponentials within each column
  sums = colSums(num)

  # Normalize each column
  result = sweep(num, 2, sums, FUN = "/")

  return(result)
}



#rm(list = ls())
```

```
X_feat=cbind(dat$X1,dat$X2)
Y=cbind(dat$Yi1,dat$Yi2,dat$Yi3)

choose_label=function(vector_prob){

  labels_vec=ifelse(vector_prob==max(vector_prob),1,0)
  return(labels_vec)

}


feedforward_nn=function(X_input,y_input,theta,m,nu,activation_fun_2=soft_max,ac

  # theta is the vector of initial weights
  #it is of length p*m+m*q+m+q (W1+W2+b1+b2)
  # in this code q=d2

  y=as.matrix(y_input)

  # Initialize relevant layer matrices


  N=dim(X_input)[1] # nr obs
  p=dim(X_input)[2] # nr of cols=nr of nodes in input
 ↪  layer for obs i
  #1. nr of nodes in input layer=2 features across N
   ↪  observations

  # Going into layer 1
  d0=p # nodes in input
  d1=m # nodes in hidden
  d2=dim(y)[2]

  X=t(as.matrix(X_input)) # obs as cols

  index_chunk1=p*m #W1
  index_chunk2=index_chunk1+1 #W2
  index_chunk3=((index_chunk2+(m*d2))-1)+1 #B1
  index_chunk4=((index_chunk3+m)-1)+1 #B2


  # weight matrix from input layer to hidden layer
```

```
 W1=matrix(theta[1:(index_chunk1)],nrow=d0,ncol=d1)

 #weight matrix from hidden layer to output layer

↪  W2=matrix(theta[(index_chunk2):((index_chunk2+(m*d2))-1)],nrow=d1,ncol=d2)
↪  #

 # bias matrix going into hidden layer

↪  b1=matrix(theta[(index_chunk3):((index_chunk3+m)-1)],nrow=m,ncol=1)
↪

 b1_ones_matrix=matrix(1,nrow=1,ncol=N)
 b1_mat=b1%*%b1_ones_matrix


 # bias matrix going  into output/layer 2

↪  b2=matrix(theta[index_chunk4:((index_chunk4+d2)-1)],nrow=d2,ncol=1)
 b2_ones_matrix=matrix(1,nrow=1,ncol=N) # Repeat this N
↪  times
 b2_mat=b2%*%b2_ones_matrix

# Do one forward calc and get prediction

 # first layer computation
 first_layer_comp=t(W1)%*%X+b1_mat

 # apply activation
 first_layer_comp=activation_fun_1(first_layer_comp)

 # second layer computation


↪  second_layer_comp=activation_fun_2(t(W2)%*%first_layer_comp+b2_mat)

↪  second_layer_comp_no_activation=t(W2)%*%first_layer_comp+b2_mat


 #predictions across N observations
 # prediction = max probability
 y_preds=second_layer_comp # 3 by N
 y_pred_label=apply(y_preds,1,choose_label) # one hot
↪  encoding
```

```
   # control for very small values in the matrix
  log_compute= function(mat, threshold = 1e-6) {
    ifelse(mat < threshold, log1p(mat), log(mat))
  }


  CE_mat=t(y)*log_compute(y_preds)
  CE_error=-mean(apply(CE_mat,1,sum)) # normal
 ↪  non_penalized oobj

  CE_error_L2=CE_error+(nu/N)*(sum(W1^2)+sum(W2^2)) #
 ↪  regularized obj



return(list(first_layer_comp =
 ↪  first_layer_comp,second_layer_comp =
 ↪  second_layer_comp, CE_error = CE_error, CE_error_L2
 ↪  =
 ↪  CE_error_L2,second_layer_comp_no_activation=second_layer_comp_no_activation


}


set.seed(2022)

y_input=Y
X_input=X_feat
#
m   = 10 # number nodes in hidden layer
p     = dim(X_input)[2]
q     = dim(as.matrix(Y))[2]

npars = p*m+m*q+m+q
theta_rand = runif(npars,-1,1)
N       = dim(X_feat)[1]
set     = sample(1:N,0.5*N,replace = FALSE) #50 50 split
X_train = X_feat[set,,drop = FALSE] #
Y_train = Y[set,,drop = FALSE]
X_val   = X_feat[-set,,drop = FALSE]
```

```
Y_val   = Y[-set,,drop = FALSE]

obj_pen = function(pars)
{
  res_model =
 ↪  feedforward_nn(X_train,Y_train,pars,m,nu,activation_fun_2=soft_max,activati
 ↪
  return(res_model$CE_error_L2) # Reg Obj
}

n_nus = 15
nu_seq= exp(seq(-2,2,length = n_nus)) # sequence of nus
 ↪  to validate against
val_error = rep(0,n_nus)
for(i in 1:n_nus)
{
    nu = nu_seq[i] # Pick nu
    res_opt = nlm(obj_pen,theta_rand, iterlim = 1000) #
 ↪  Fit for that nu

    #Evaluate on Validation Set with optimal parameters
     ↪  and particular nu

    res_val=
 ↪  feedforward_nn(X_val,Y_val,res_opt$estimate,m,nu,activation_fun_2=soft_max,

    # Get reg validation Error
  val_error[i] = res_val$CE_error_L2

  #print(paste0('Validation run:',i,' nu = ', nu))
}


#| fig-dpi: 300


min_val_error=min(val_error)



best_nu_index=which(val_error==min(val_error))
best_nu=nu_seq[best_nu_index];
```

```
# Compare the error of the nu the right of the best

next_val_error=val_error[(best_nu_index+1):length(val_error)]

next_seq=nu_seq[(best_nu_index)+1:length(nu_seq)]

next_best_nu=next_seq[which.min(next_val_error)] ;

next_min_val_error=min(next_val_error)



plot(val_error~log(nu_seq), type = 'b',pch =
 ↪  16,xlim=c(min(log(nu_seq)),max(log(nu_seq))+1),
 ↪  main='Validation Error vs Regularization Parameter
 ↪  (nu)', xlab='lambda (log)',ylab='validation error')

abline(v =log(best_nu), col = "purple", lty = "dashed",
 ↪  lwd = 2)

abline(v =log(next_best_nu), col = "blue", lty =
 ↪  "dashed", lwd = 2)

text(x = log(best_nu)+0.33, y = 30, labels = paste("best
 ↪  nu:",round(best_nu,3), 'Val
 ↪  Error:',round(min_val_error,2),sep = "\n"),
     srt = 0, adj = c(0.5, -0.5), col =
      ↪  "purple",cex=0.68)


text(x = log(next_best_nu)+0.55, y = 30, labels =
 ↪  paste("next best nu:",round(next_best_nu,3), 'Val
 ↪  Error:',round(next_min_val_error,2),sep = "\n"),
     srt = 0, adj = c(0.5, -0.5), col = "blue",cex=0.68)
#
#


library(colorspace)

# Fit probability color mapping for each class onto the
 ↪  feature space
```

```
# Helper functions

# Assign color pallet based on indexing
  ass_pallet=function(ind_val,col_steps=50){
    if(ind_val==1){

      # Class 1, high to low prob color mapping
      pallet=colorRampPalette(c("lightblue",
 ↪  'white',"darkblue"))

    }

    if(ind_val==2){

      # Class 2
      pallet=colorRampPalette(c("mistyrose",'white',
 ↪  "darkred"))

    }
   if(ind_val==3){

     # Class 3
      pallet=colorRampPalette(c("mediumpurple1",'white',
 ↪  "purple4"))

   }

    return(pallet(col_steps))
  }


# Assign a color to prediction based on its class and
 ↪  strength of probability value

col_pred=function(x,col_steps=50){
  # x=prediction matrix from ff nn

  max_ind=apply(x,2,which.max) #get index of max
 ↪  probability
  max_prob=apply(x,2,max) # get value of max probability
```

```
  col_range=seq(0,1,length.out = col_steps)
  col_pallet=sapply(max_ind,ass_pallet) #prob colors for
 ↪  each class, matrix
  color_indices= findInterval(max_prob, col_range) # for
 ↪  each obs, find its indx in a range of 0 to 1

  col_pallet_N=ncol(col_pallet)
  selected_cols= col_pallet[cbind(color_indices,
 ↪  seq_len(col_pallet_N))] # get appropriate index in
 ↪  the pallette assignment matrix

  return(selected_cols)
}

pal1 = ass_pallet(1)
pal2 = ass_pallet(2)
pal3 = ass_pallet(3)

obj_pen = function(pars)
{
  res_model
 ↪  =feedforward_nn(X_input,y_input,pars,m,next_best_nu,activation_fun_2=soft_m
  return(res_model$CE_error_L2) # Reg Obj
}

# Optimize weights using next best nu
res_opt_best_nu = nlm(obj_pen,theta_rand, iterlim =
 ↪  1000)



M   = 160
x1  = seq(-5,+5,length = M)
x2  = seq(-5,+5,length = M)
xx1 = rep(x1,M)
xx2 = rep(x2,each = M)

layout(matrix(1:4, nrow = 1),
       widths = c(5, 0.7, 0.7, 0.7))

par(mar = c(4, 4, 3, 1))

plot(NULL,
 ↪  xlim=c(-5,5),ylim=c(-5,5),xlab='x1',ylab='x2',
 ↪  main='Feed Forward NN Response Curve')
```

```
abline(h= x2,v = x1, lwd = 0.3,col='black')



XX = cbind(xx1,xx2)
YY1 = rep(1,M^2)
YY2=rep(1,M^2)
YY3=rep(1,M^2)
YY=cbind(YY1,YY2,YY3)


# Again use next best nu
res_fitted=feedforward_nn(XX,YY,res_opt_best_nu$estimate,m,next_best_nu,activat
 ↪

points(xx2~xx1,pch = 16,
 ↪  col=col_pred(res_fitted$second_layer_comp) ,cex = 2)

points(dat[dat$Yi1==1,]$X1,dat[dat$Yi1==1,]$X2,col='blue',pch=10)

points(dat[dat$Yi2==1,]$X1,dat[dat$Yi2==1,]$X2,col='red',pch=10)

points(dat[dat$Yi3==1,]$X1,dat[dat$Yi3==1,]$X2,col='purple',pch=10)

add_colour_bar = function(pal, class_name) {

  par(mar = c(4, 0.5, 3, 2))

  probs <- seq(0, 1, length.out = length(pal))

  # Draw continuous-looking gradient
  z = matrix(rep(probs, each = 2), nrow = 2)

  image(x = c(0, 1),
        y = probs,
        z = z,
        col = pal,
        zlim = c(0, 1),
        axes = FALSE,
        xlab = "",
        ylab = "",
```

```
        main = class_name)

  axis(4, at = seq(0, 1, by = 0.2),
       las = 1, cex.axis = 0.75)

  box()
}

#colour mapping
add_colour_bar(pal1, "Class 1")
add_colour_bar(pal2, "Class 2")
add_colour_bar(pal3, "Class 3")
```